Delving into these heuristics is out of the scope of this section, we refer the curious reader to the original paper by Antonin Guttman for a proper description; at this point, though, we can already reveal that the complexity of handling hyper-rectangles and obtain good splits (and merges, after removals) is one of the main reasons that lead to the introduction of SS-trees.

10.2.4 Search

Searching for a point, or for the nearest neighbor (*NN*) of a point, in R-trees is very much similar to what happens in k-d trees: we need to traverse the tree, pruning branches that can't contain a point or, for NN search, are certainly further away than the current minimum distance.

Figure 10.8 shows an example of (unsuccessful) point search on our example R-tree: remember that unsuccessful search is the first step for inserting a new point, through which we can find the rectangle (or rectangles, like this case) where we should add the new point.

The search starts at the root, where we compare point P's coordinates with the boundaries of each rectangle, R1 and R2: P can only be within R2, so this is the only branch we traverse.

At the next step, we go through R2's children, R5 and R6: both can contain P, so we need to traverse both branches at this level (as shown by the two red arrows leaving R2 on the left side of figure 10.8).